

Cleaning_Data

August 15, 2022

```
[ ]: !pip install import-ipynb
```

```
[ ]: # importing the tools to call google drive
from google.colab import drive
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import import_ipynb
import pickle as.pkl
```

```
[ ]: # mount our directory
mount='/content/gdrive'
drive.mount(mount, force_remount=True)
```

Mounted at /content/gdrive

```
[ ]: # Switch to the directory on the VM for our purposes
drive_root=mount + '/My Drive/Colab Notebooks/Thesis'
%cd $drive_root
%pwd
```

/content/gdrive/My Drive/Colab Notebooks/Thesis

```
[ ]: '/content/gdrive/My Drive/Colab Notebooks/Thesis'
```

```
[ ]: import Pitch_Plot
import data
from data import obtain_seasons,fetch_url,Game
from Pitch_Plot import pitchplot
```

importing Jupyter notebook from Pitch_Plot.ipynb
importing Jupyter notebook from data.ipynb

```
[ ]: laliga = obtain_seasons(11)
```

```
[ ]: pitchplot()
goalkeeper_x = 120
goalkeeper_y = 40
player_list_x = []
```

```

player_list_y = []
player_name = []
team_name = []
location_of_shot_x = 0
location_of_shot_y = 0
shot_end_location_x = 0
shot_end_location_y = 0
teammate_x = []
teammate_y = []
for events in laliga["2020/2021"]['3764661'].json_file:
    if events['id'] == "1d9a647b-7207-4425-881c-ac532115239a":
        location_of_shot_x = events['location'][0]
        location_of_shot_y = events['location'][1]
        shot_end_location_x = events['shot']['end_location'][0]
        shot_end_location_y = events['shot']['end_location'][1]

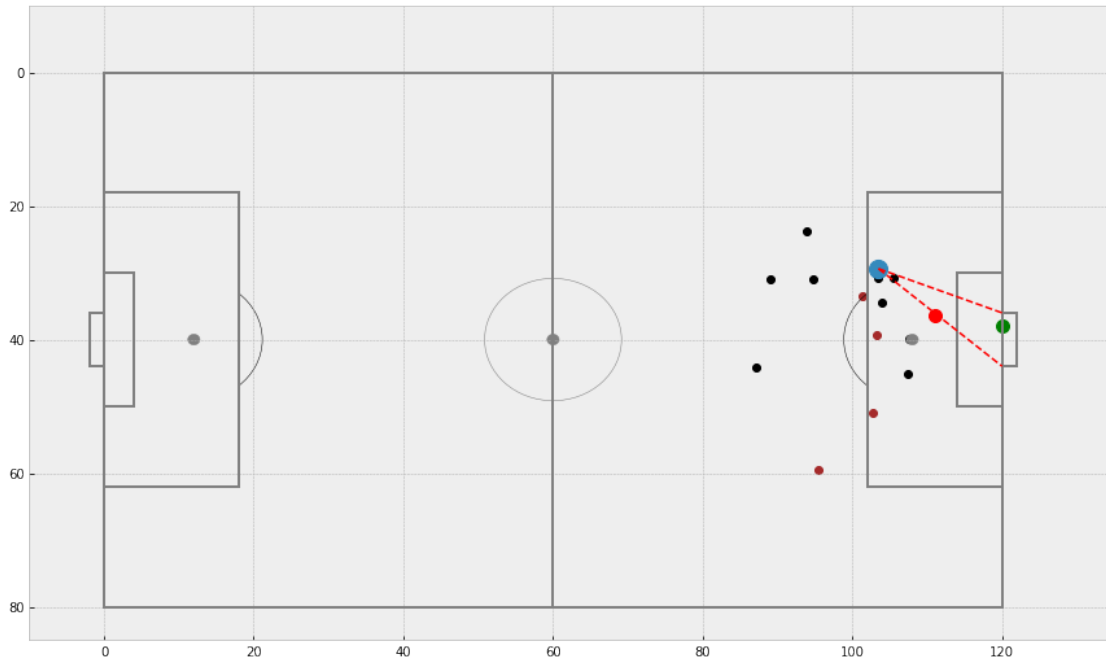
        if "freeze_frame" in events["shot"]:
            for players in events['shot']['freeze_frame']:
                player_name.append(events['player']['name'])
                if (players['teammate'] == False):
                    player_list_x.append(players['location'][0])
                    player_list_y.append(players['location'][1])
                    team_name.append(events['possession_team']['name'])
                elif (players['teammate'] == True):
                    teammate_x.append(players['location'][0])
                    teammate_y.append(players['location'][1])
                if ((players['position']['name'] == 'Goalkeeper') and
→(players['teammate'] == False)):
                    goalkeeper_x = players['location'][0]
                    goalkeeper_y = players['location'][1]

plt.scatter(player_list_x, player_list_y, color = 'black')
plt.scatter(teammate_x, teammate_y, color = 'brown')
plt.scatter(location_of_shot_x, location_of_shot_y, s=200)
plt.scatter(goalkeeper_x, goalkeeper_y, s=100, color='red')
plt.scatter(shot_end_location_x, shot_end_location_y, s=100, color='green')
plt.plot([location_of_shot_x, 120], [location_of_shot_y,36], color = 'red',
→linestyle = 'dashed')
plt.plot([location_of_shot_x, 120], [location_of_shot_y,44], color = 'red',
→linestyle = 'dashed')
plt.title(player_name[0] + '\n' + team_name[0], fontsize = 30)

```

[]: Text(0.5, 1.0, 'Lionel Andrés Messi Cuccittini\nBarcelona')

Lionel Andrés Messi Cuccittini Barcelona



```
[ ]: def player_density(x_shot, y_shot, x_player, y_player):
    x_difference = x_player - x_shot
    y_difference = y_player - y_shot

    if 120 - x_shot == 0:
        return False

    slope_1 = (36 - y_shot) / (120 - x_shot)
    slope_2 = (44 - y_shot) / (120 - x_shot)

    return (x_difference >= 0) and ((y_shot + slope_1 * x_difference) < y_player
    ↪ (y_shot + slope_2 * x_difference))
```

```
[ ]: def pass_type(id, match):
    """
    This function takes the id of the goal assist and returns which type of pass
    ↪ it is.

    Arguments:
    id: id of the goal assist
    match: details of the match file
    """
    key_pass_type = ''
```

```

for events in match:
    if events['id'] == id:
        key_pass_type = events['pass']['height']['name']
        break
return key_pass_type

```

```

[ ]: def obtain_shots_match(match):
    """
    Function which parses through a game json file and returns a data frame
    → containing all shots taken in that game with several features
    related to that shot

    Arguments
    game_json - details of the json file
    """

    #features for each shot, which will be the columns of our data frame
    shot_id_list = []
    x_start_location_list = []
    y_start_location_list = []
    play_pattern_list = []
    duration_list = []
    outcome_list = []
    technique_list = []
    type_shot_list = []
    first_time_list = []
    body_part_name = []
    x_gk_pos_list = []
    y_gk_pos_list = []
    num_opponents_5_yards_list = []
    num_opponents_between_goal_list = []
    statsbomb_xg = []
    team_name = []
    player_name = []
    key_pass_id = []
    key_pass_type = []
    #-----#

    for events in match:

        if events['type']['name'] == 'Shot':

            #get data immediately needed features
            shot_id = events['id']
            x_start_location = events['location'][0]
            y_start_location = events['location'][1]
            play_pattern = events['play_pattern']['name']

```

```

duration = events['duration']
outcome = events['shot']['outcome']['name']
technique = events['shot']['technique']['name']
type_shot = events['shot']['type']['name']
xg = events['shot']['statsbomb_xg']
body_part = events['shot']['body_part']['name']
team = events['possession_team']['name']
player1 = events['player']['name']
pass_id = ''
type_pass = ''

try:
    pass_id = events['shot']['key_pass_id']
    type_pass = pass_type(pass_id, match)

except KeyError:
    pass_id = 'No Pass'
    type_pass = 'None'

    #check if json shot has a first_time attribute, if not set
    →first_time to False
    if 'first_time' in events['shot']:
        first_time = events['shot']['first_time']

    else:
        first_time = False

    #check if shot has a freeze_frame dictionary
    if "freeze_frame" in events["shot"]:

        num_opponents_5_yards = 0
        num_opponents_between_goal = 0

        for player in events["shot"]["freeze_frame"]:
            x_player = player['location'][0]
            y_player = player['location'][1]

            #count how many opponents were within 5 yards of player when
            →shot was taken
            if ((x_start_location - x_player)**2 + (y_start_location -
            →y_player)**2) <= 25 and (player['teammate'] == False):
                if (player['position']['name'] != 'Goalkeeper'):
                    num_opponents_5_yards += 1

            #count how many opponents were between shot and goal
            if (player['teammate'] == False):

```

```

        if player_density(x_start_location, y_start_location,
→x_player, y_player):
            num_opponents_between_goal += 1

            #get position of opponent's goalkeeper
            if ((player['position']['name'] == 'Goalkeeper') and
→(player['teammate'] == False)):
                x_gk_pos = player['location'][0]
                y_gk_pos = player['location'][1]

            #if there is no freeze frame, assume goalkeeper is at center of
→goal, and 0 opponenets around shot location
            else:
                num_opponents_between_goal = 0
                num_opponents_5_yards = 0
                x_gk_pos = 120
                y_gk_pos = 40

            try:
                x_gk_pos_list.append(x_gk_pos)
                y_gk_pos_list.append(y_gk_pos)

            except UnboundLocalError:
                x_gk_pos_list.append(np.nan)
                y_gk_pos_list.append(np.nan)

        #append data on shot to relevant list (column)
        shot_id_list.append(shot_id)
        play_pattern_list.append(play_pattern)
        x_start_location_list.append(x_start_location)
        y_start_location_list.append(y_start_location)
        duration_list.append(duration)
        outcome_list.append(outcome)
        technique_list.append(technique)
        first_time_list.append(first_time)
        type_shot_list.append(type_shot)
        body_part_name.append(body_part)
        num_opponents_5_yards_list.append(num_opponents_5_yards)
        num_opponents_between_goal_list.append(num_opponents_between_goal)
        statsbomb_xg.append(xg)
        team_name.append(team)
        player_name.append(player1)
        key_pass_id.append(pass_id)
        key_pass_type.append(type_pass)

        #create a data frame with the features

```

```

shot_df = pd.DataFrame({
    "shot id" : shot_id_list,
    "play pattern" : play_pattern_list,
    "x location shot" : x_start_location_list,
    "y location shot" : y_start_location_list,
    "duration" : duration_list,
    "outcome" : outcome_list,
    "technique used" : technique_list,
    "first time" : first_time_list,
    "x gk location" : x_gk_pos_list,
    "y gk location" : y_gk_pos_list,
    "body part" : body_part_name,
    "type of shot" : type_shot_list,
    "Number of opponents in 5 yards" :
↪ num_opponents_5_yards_list,
    "Players between goal" : num_opponents_between_goal_list,
    "player name" : player_name,
    "Team Name" : team_name,
    "official xg" : statsbomb_xg,
    "Pass id": key_pass_id,
    "Pass Type": key_pass_type
})

return shot_df

```

```

[ ]: def obtain_season_shots(season):
    """
    Takes a dictionary whichs maps game ids to Game objects, and calls
↪ obtain_shots_match() on each one.

    Arguments:
    season: dictionary which has game file names (string, as specified by
↪ statsbomb) to objects (defined above)
    """
    total_shot_df = pd.DataFrame()

    for keys, values in season.items():
        shot_df = obtain_shots_match(values.json_file)
        shot_df["game_id"] = keys
        total_shot_df = total_shot_df.append(shot_df)

    return total_shot_df

def obtain_all_shots(league):
    """

```

Takes the original data from statsbomb and passes it to obtain_season_shots_
→to process and procure football shots.

Arguments:

league - The dictionary that iterates season names to a further dictionary.
→The latter iterates

match ids (as defined by statsbomb) to all events in the matches.

"""

```
total_shot_df = pd.DataFrame()
```

```
for keys, values in league.items():
    shot_df = obtain_season_shots(values)
    shot_df["season_id"] = keys
    total_shot_df = total_shot_df.append(shot_df)
    print("Obtaining shots for " + keys)
```

```
print("Done.")
```

```
return total_shot_df
```

```
[ ]: with open('laliga.pickle', 'wb') as handle:
      pickle.dump(laliga, handle, protocol=pickle.HIGHEST_PROTOCOL)
```

```
[ ]: with open('laliga.pickle', 'rb') as handle:
      b = pickle.load(handle)
```

```
[ ]: laliga_shots = obtain_all_shots(laliga)
```

```
[ ]: laliga_shots['distance_from_goalpost_a'] = np.sqrt((laliga_shots['x location_
→shot']-120)**2+(laliga_shots['y location shot']-36)**2)
laliga_shots['distance_from_goalpost_b'] = np.sqrt((laliga_shots['x location_
→shot']-120)**2+(laliga_shots['y location shot']-44)**2)

laliga_shots['shot_angle'] = np.
→arccos((laliga_shots['distance_from_goalpost_a']**2 +_
→laliga_shots['distance_from_goalpost_b']**2 - 64)/
→(2*laliga_shots['distance_from_goalpost_a']*laliga_shots['distance_from_goalpost_b']))
laliga_shots['shot_angle'] = np.degrees(laliga_shots['shot_angle'])
laliga_shots['shot_angle'] = round(laliga_shots['shot_angle'],1)
```

/usr/local/lib/python3.7/dist-packages/pandas/core/arraylike.py:364:

RuntimeWarning: invalid value encountered in arccos

result = getattr(ufunc, method)(*inputs, **kwargs)

```
[ ]: laliga_shots.isna().sum()
```

```
[ ]: shot id          0
      play pattern    0
      x location shot 0
      y location shot 0
      duration        0
      outcome         0
      technique used  0
      first time      0
      x gk location   0
      y gk location   0
      body part       0
      type of shot    0
      Number of opponents in 5 yards 0
      Players between goal 0
      player name     0
      Team Name       0
      official xg     0
      Pass id         0
      Pass Type       0
      game_id         0
      season_id       0
      distance_from_goalpost_a 0
      distance_from_goalpost_b 0
      shot_angle      1
      dtype: int64
```

```
[ ]: laliga_shots[laliga_shots['shot_angle'].isna()]
```

```
[ ]:          shot id    play pattern  x location shot \
24  7540f106-869b-4cb8-8a6b-d1db41f7b759  From Free Kick      120.0

      y location shot  duration outcome technique used  first time \
24          43.7    0.3009   Goal          Normal        True

      x gk location  y gk location  ...          player name  Team Name \
24          113.2          33.3  ...  Luis Alberto Suárez Díaz  Barcelona

      official xg  Pass id Pass Type game_id  season_id \
24    0.898411  No Pass      None  266254  2015/2016

      distance_from_goalpost_a distance_from_goalpost_b shot_angle
24          7.7          0.3      NaN

[1 rows x 24 columns]
```

```
[ ]: laliga_shots['shot_angle']=laliga_shots['shot_angle'].fillna(0)
```

```
[ ]: laliga_shots.isna().sum()
```

```
[ ]: shot id          0
      play pattern    0
      x location shot 0
      y location shot 0
      duration        0
      outcome         0
      technique used   0
      first time       0
      x gk location    0
      y gk location    0
      body part        0
      type of shot     0
      Number of opponents in 5 yards 0
      Players between goal 0
      player name      0
      Team Name        0
      official xg      0
      Pass id          0
      Pass Type        0
      game_id          0
      season_id        0
      distance_from_goalpost_a 0
      distance_from_goalpost_b 0
      shot_angle       0
      dtype: int64
```

```
[ ]: laliga_shots.to_pickle('./laliga_data.pkl')
```